ADITYA UNIVERSITY

## School of Computing
## Department of Computer Science and Engineering
## MACHINE LEARNING

# CERTIFICATE

This is to certify that the project work entitled **"Sentiment Analysis Using Ensemble Learning in Machine Learning"** is submitted by

**K. Arjun (25B21CS078)**
**K. Lokesh (25B21CS080)**
**M. Durga Prasad (25B21CS079)**
**N. Kalyan Venkat Reddy (25B21CS075)**

in partial fulfilment of the requirements for the award of the B.Tech. degree in Computer Science and Engineering for the academic year 2026-2027.

Project Guide
Dr. B. Sindhu
Assistant Professor
Department of CSE
Aditya University

Head of the Department
Dr. A. Phani Sridhar
Assistant Professor & HOD
Department of CSE
Aditya University

# DECLARATION

We hereby declare that the project entitled **"Sentiment Analysis Using Ensemble Learning in Machine Learning"** is original and is submitted to Aditya University, Surampalem in partial fulfilment of the requirements for the award of the B.Tech. degree in Computer Science and Engineering. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy.

**Place:** Surampalem

**Date: 12-10-2026**

By

K. Arjun (25B21CS078)

K. Lokesh (25B21CS080)

M. Durga Prasad (25B21CS079)

N. Kalyan Venkat Reddy (25B21CS075)

# ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Project Guide**, Dr. B. Sindhu, Assistant Professor**, who has guided us throughout the project. Their valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our **Deputy Heads of the Department, Dr. Hari J, Associate Professor, Dr. Siva Satya Prasad P, Assistant Professor, and Dr. Chandra Sekhar K, Associate Professor, Department of CSE**, for their support and valuable suggestions.

Our deepest thanks to our **Head of the Department, Dr. A. Phani Sridhar, Assistant Professor & HOD**, for the inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Computing; Dr. G. Suresh, Registrar; Dr. A. Ramesh, Pro Vice-Chancellor** (Engineering & Sciences); **Dr. S. Rama Sree, Pro ViceChancellor** (Academics); **Dr. M.B. Srinivas, Vice-Chancellor; and Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management**, Aditya University, for their encouragement and support during the course of our project.

Finally, we would like to thank the faculty members, lab technicians, non-teaching staff, and our friends who have directly or indirectly supported us in completing this project successfully.

# ABSTRACT

Sentiment analysis identifies the opinion or emotion expressed in text. Social media holds a huge amount of opinion-based text, and reading it manually is slow, while existing lexicon-based methods and single classifiers struggle with slang, spelling variations, sarcasm and noisy text. This project classifies tweets as Positive, Negative or Neutral using the Twitter Entity Sentiment Analysis dataset from Kaggle, which contains 74,682 records. The tweet text is first cleaned (lowercasing, removal of missing tweets, URLs, user mentions and unwanted characters, and normalisation of spaces), leaving 61,121 records, and is then converted into numbers using TF-IDF. Both Word-level and Character-level features are used so that informal language and spelling variations are captured. A Random Forest, an ensemble of decision trees whose predictions are combined by majority vote, is trained on these features, and a hybrid approach combines rule-based analysis with the machine learning prediction. The trained model and vectorizers are saved with Joblib and used through a Flask web interface in which the user enters a tweet or sentence and receives the predicted sentiment. The model achieved an accuracy of 90.94% on 12,225 unseen test tweets (20% of the data), with macro-averaged precision, recall and F1-score of 0.91. The system was also tested with new text: “I really love this game” was classified as Positive, “I hate this product” as Negative and “The game is okay” as Neutral, and the mixed-sentiment sentence “product is good but not working” as Negative. The results show that an ensemble-based, hybrid approach is a practical way to classify the sentiment of short, informal text.



**Keywords:** Sentiment Analysis, Ensemble Learning, Machine Learning, Text Classification, Natural Language Processing, Feature Extraction, Model Evaluation

# LIST OF CONTENTS

LIST OF FIGURES ..... i
Chapter 1: Introduction ..... 01
1.1 Introduction ..... 01
1.2 Problem Statement ..... 01
1.3 Objectives of the Project ..... 02
1.4 Scope of the Project ..... 02
1.5 Importance of Sentiment Analysis ..... 03
1.6 Motivation ..... 03
1.7 Applications of Sentiment Analysis ..... 03
1.8 Literature Review ..... 03
Chapter 2: System Overview ..... 06
2.1 Existing System ..... 06
2.2 Limitations of Existing System ..... 06
2.3 Proposed System ..... 06
2.4 Software Requirements ..... 06
2.5 Hardware Requirements ..... 07
2.6 Advantages of the Proposed System ..... 07
2.7 Functional Requirements ..... 08
2.8 Non-Functional Requirements ..... 08
Chapter 3: System Implementation ..... 09
3.1 Overview of Development Tools ..... 09
3.2 Dataset Description ..... 09
3.3 Implementation Details ..... 10
Chapter 4: System Architecture ..... 12
4.1 System Architecture ..... 12
4.2 Workflow of the System ..... 13
4.3 Data Flow Diagram ..... 13
4.4 Dataset Processing Pipeline ..... 14
4.5 Machine Learning Model and Ensemble Architecture ..... 14
4.6 Inputs and Outputs of the Stages ..... 15
Chapter 5: Data Preprocessing ..... 17
5.1 Introduction ..... 17
5.2 Handling Missing Values ..... 17
5.3 Data Cleaning ..... 17
5.4 Outlier Detection and Range Validation ..... 19
5.5 Feature Engineering / Encoding ..... 19
5.6 Data Visualization ..... 19
5.7 Train-Test Split ..... 20
5.8 Summary ..... 20
Chapter 6: Methodology and Algorithms ..... 21
6.1 Introduction ..... 21
6.2 Classification Techniques ..... 21
6.3 Ensemble and Hybrid Technique ..... 22
6.4 Model Training and Output ..... 22
6.5 Model Evaluation Metrics ..... 23
Chapter 7: Results and Analysis ..... 25
7.1 Introduction ..... 25
7.2 Experimental Setup ..... 25

7.3 Random Forest Results ..... 25
7.4 Hybrid Rule-Based System Results ..... 26
7.5 Comparison of Models ..... 27
7.6 Key Findings ..... 27
7.7 Testing ..... 27
Chapter 8: Output Screens ..... 31
8.1 Web Interface ..... 31
8.2 Sample Prediction Outputs ..... 31
Chapter 9: Conclusion ..... 33
9.1 Summary of the Project ..... 33
9.2 Achievement of Objectives ..... 33
9.3 Main Findings ..... 33
9.4 Current Limitations ..... 34
Chapter 10: Future Scope ..... 35
Chapter 11: References ..... 36
Appendix A: Source Code ..... 36

# LIST OF FIGURES

Fig 1.1: Problem Statement and Proposed Solution
Fig 3.1: Sample Rows of the Dataset (Tweet ID, Entity, Sentiment, Tweet)
Fig 4.1: System Architecture of the Sentiment Analysis System
Fig 4.2: Workflow of the Sentiment Analysis System
Fig 4.3: Level-0 Data Flow Diagram
Fig 4.4: Data Processing Pipeline
Fig 4.5: Random Forest and Hybrid Ensemble Architecture
Fig 5.1: Text Preprocessing Steps with an Example Tweet
Fig 5.2: Dataset Cleaning Summary
Fig 5.3: Combination of Word-Level and Character-Level TF-IDF Features
Fig 5.4: Class Distribution of the Cleaned Dataset
Fig 5.5: Train–Test Split of the Cleaned Dataset
Fig 5.6: Number of Records at Each Stage of the Pipeline
Fig 6.1: Output Screen of the Training Script (train_model.py)
Fig 7.1: Per-Class Precision, Recall and F1-Score of the Random Forest
Fig 7.2: Confusion Matrix of the Random Forest Model
Fig 8.1: Web Interface of the Sentiment Analysis System
Fig 8.2: Prediction Output for “i love this product” (Positive)
Fig 8.3: Prediction Output for “product is good but not working” (Negative)

# CHAPTER 1

# INTRODUCTION

## 1.1 Introduction

Sentiment analysis, also called opinion mining, is the task of automatically determining the attitude expressed in a piece of text, such as whether the writer is positive, negative or neutral towards a product, a service, a game or an event [1], [2]. People now publish an enormous amount of opinionated text every day on social media platforms, review sites and discussion forums. Reading and judging this volume of text manually is not practical, so computational methods that classify sentiment automatically have become an important area of Natural Language Processing (NLP) and Machine Learning (ML).

Sentiment in short, informal text such as tweets is difficult to classify. Such text is brief and contains spelling variations, slang, abbreviations, missing punctuation and mixed opinions within a single sentence. A single classifier trained on such data may perform well on some kinds of sentences and poorly on others. Ensemble learning addresses this by combining the predictions of several classifiers so that the errors of one model can be offset by the others [8], [13].

This project, **Sentiment Analysis Using Ensemble Learning in Machine Learning**, classifies tweets into Positive, Negative and Neutral sentiment using the Twitter Entity Sentiment Analysis dataset from Kaggle. The tweet text is cleaned and preprocessed and converted into numbers using Word-level and Character-level TF-IDF features, so that informal language and spelling variations are captured. A Random Forest, which is an ensemble of many decision trees, is trained on these features. A hybrid approach then combines rule-based analysis with the machine learning prediction. The result is shown through a Flask web interface in which the user enters a tweet or sentence and sees the predicted sentiment.

## 1.2 Problem Statement

Manual reading of opinions does not scale, and lexicon-based or single-model approaches can be sensitive to the writing style, vocabulary and noise of the data on which they were built. A sentence such as “product is good but not working” contains both positive and negative expressions, and a single classifier may be misled by one of them. An automated system is required that learns sentiment patterns from labelled text and gives a more dependable prediction by combining several models.

The problem addressed in this project is therefore framed as a supervised text-classification problem: given a tweet or sentence, represented by a numerical feature vector derived from its words and characters, predict its sentiment class (Positive, Negative or Neutral) using an ensemble of decision trees (Random Forest) supported by rule-based analysis. Fig 1.1 summarises the problem and the proposed solution.

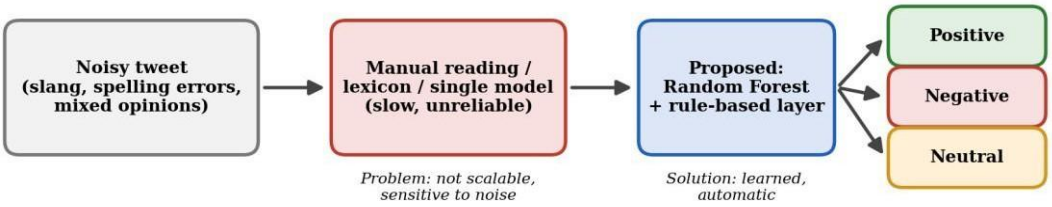



**Fig 1.1:** Problem Statement and Proposed Solution

## 1.3 Objectives of the Project

- **Dataset collection** – collect and prepare the Twitter sentiment dataset.
- **Preprocessing** – clean and preprocess the tweet text.
- **Feature extraction** – convert the text into numbers using TF-IDF.
- **Word and character features** – use both Word-level and Character-level TF-IDF features.
- **Model training** – train a Random Forest classifier.
- **Prediction** – predict the sentiment of unseen text.
- **Hybrid improvement** – improve the prediction with rule-based analysis.
- **Interface** – provide a simple interface to enter text and see the result.

## 1.4 Scope of the Project

The scope below separates what has been carried out in the present work from what is proposed only as a future enhancement (see Chapter 10).

**Table 1.1:** Current Scope and Future Scope of the Project

| Aspect | Current Project Scope (implemented) | Future Scope (not implemented) |
|---|---|---|
| Data | Twitter Entity Sentiment Analysis dataset (Kaggle), 74,682 records; 61,121 cleaned records in the classes Positive, Negative and Neutral | Larger and more recent datasets; other domains and languages |
| Features | Word-level and Character-level TF-IDF combined into one feature set of 15,000 features | Word embeddings and contextual embeddings |
| Algorithms | Random Forest (ensemble of decision trees) with a hybrid rule-based analysis layer | Comparison with other classifiers; voting or stacking ensembles; deep learning models |
| Evaluation | 80/20 train-test split; accuracy, precision, recall, F1-score on 12,225 test tweets; testing with new, unseen sentences | Cross-validation and statistical significance testing |
| Interface | Flask web interface (HTML and CSS) with a text box, an Analyze Sentiment button, example inputs and a prediction card | Confidence display, batch analysis, public deployment |

The present project is limited to classifying the sentiment of a single text at a time. Analysis of images, audio or video, and real-time streaming from social media platforms, are not part of the implemented system.

## 1.5 Importance of Sentiment Analysis

Automatic sentiment analysis is valuable because it converts large amounts of unstructured opinion text into information that can be acted on. Its importance can be summarised as follows.

- **Scale** – thousands or millions of texts can be processed in the time a person would need to read a few.
- **Consistency** – a trained model applies the same decision rule to every text, which reduces the variation between human readers.
- **Timeliness** – opinions can be monitored as they appear, so that problems or trends are noticed early.
- **Decision support** – organisations can use aggregated sentiment to understand customer satisfaction, public reaction and product reception.

## 1.6 Motivation

Individual classifiers have different strengths and weaknesses, and no single algorithm is best for every dataset. Ensemble learning is motivated by the observation that a combination of diverse classifiers can be more accurate and more stable than its members when their errors are not fully correlated [8]. Applying this idea to sentiment classification of short social-media text, together with a simple web interface through which anyone can test the model, motivated the selection of this project.

## 1.7 Applications of Sentiment Analysis

- Monitoring public opinion about products, brands, games and services on social media.
- Analysing customer reviews and feedback to identify satisfaction and complaints.
- Tracking reaction to events, campaigns or announcements.
- Supporting customer-support teams by prioritising negative messages.
- Market research and competitor analysis based on opinion trends [2].

## 1.8 Literature Review

### 1.8.1 Overview of Sentiment Analysis Research

Research on sentiment analysis grew rapidly in the early 2000s with the availability of online reviews and social media. The survey by Pang and Lee [1] and the book by Liu [2] give comprehensive overviews of the field, covering the classification of documents, sentences and aspects as positive, negative or neutral. Approaches can broadly be grouped into lexicon-based methods, traditional machine learning methods, ensemble methods and deep learning methods.

### 1.8.2 Traditional Machine Learning Approaches

Pang, Lee and Vaithyanathan [3] applied Naive Bayes, maximum entropy and Support Vector Machine classifiers to the polarity classification of movie reviews and reported that machine learning methods performed better than human-proposed word-list baselines, with word-presence features working well. Support Vector Machines were introduced by Cortes and Vapnik [7] and have since been widely used for text classification.

For short social-media text, Go, Bhayani and Huang [4] trained Naive Bayes, maximum entropy and SVM classifiers on tweets whose labels were obtained automatically from emoticons (distant supervision), showing that supervised learning could be applied to Twitter data without manual labelling. Rule-based lexicon methods such as VADER [5] were designed specifically for social-media text and require no training, but depend on a fixed vocabulary of sentiment-bearing words. In most of these methods the text is converted into numerical features such as word counts or term frequency–inverse document frequency (TF-IDF) weights, whose idea of inverse document frequency originates from the work of Jones [6].

### 1.8.3 Ensemble Learning Approaches

Ensemble learning combines several base learners to obtain a predictor that is usually more accurate than any single member. Dietterich [8] explained that ensembles can help for statistical, computational and representational reasons, and that the members must be reasonably accurate and diverse. Four families of ensemble methods are commonly distinguished [13]:

- **Voting** – the predictions of several different classifiers are combined by majority vote (hard voting) or by averaging predicted probabilities (soft voting).
- **Bagging** – many versions of one learner are trained on bootstrap samples and aggregated [9]; Random Forest [10] extends this idea to decision trees with random feature selection.
- **Boosting** – learners are trained one after another, each concentrating on the examples that earlier learners handled badly, as in AdaBoost [11].
- **Stacking** – the outputs of base learners are used as inputs to a further meta-learner [12].

Ensemble methods have been applied to sentiment classification by combining classifiers trained on the same text features, and the choice of members and combination rule differs between studies. The technique used in the present project is described in Chapter 6.

### 1.8.4 Comparison of Related Methods

**Table 1.2:** Comparison of Approaches to Sentiment Analysis

| Approach | Example / Source | Main Strength | Main Limitation |
|---|---|---|---|
| Lexicon / rule based | VADER [5] | No training data needed; designed for social-media text | Limited to the words and rules in the lexicon |
| Single ML classifier | Naive Bayes, MaxEnt, SVM [3], [4] | Learns from labelled data; simple and fast | Performance depends on one model and on the chosen features |
| Ensemble ML | Bagging, boosting, voting, stacking [8]–[13] | Combines several models; can reduce variance and error | More models to train; gain is not guaranteed |
| Deep learning | CNN [14], BERT [15] | Learns features directly from text; strong results on many benchmarks | Needs more data and computing power; harder to interpret |

### 1.8.5 Research Gaps and Motivation

Single classifiers can be unstable on short, noisy social-media text, while deep learning models require substantial data and computing resources. An ensemble of classical machine learning models offers a middle path: it keeps training inexpensive and the results easy to explain, while combining several viewpoints on the same text. The motivation for the proposed approach is therefore to examine how an ensemble behaves on a labelled sentiment dataset and to make it available to users through a simple web interface. Whether the ensemble outperforms other individual models must be judged from measured comparisons and is not assumed in advance.

The reviewed work also shows that the quality of the features matters as much as the choice of classifier. Wordlevel TF-IDF features capture the vocabulary of positive and negative opinions, while character-level features make the representation less sensitive to misspellings, elongated words and other irregularities that are common in tweets. The present project therefore combines both representations and uses a Random Forest as a robust, easily trained ensemble on top of them.

# CHAPTER 2

# SYSTEM OVERVIEW

## 2.1 Existing System

Existing systems for judging the sentiment of tweets mostly use the following approaches.

- **Manual reading** – a person reads each tweet and decides its sentiment; this is slow and difficult for thousands of tweets.
- **Lexicon-based methods** – words are matched against a list of sentiment-bearing words and simple rules decide the label [5].
- **Single-classifier models** – one machine learning model, such as Naive Bayes or Logistic Regression, is trained on labelled text and used for prediction [3], [4].

## 2.2 Limitations of Existing System

- It is difficult to detect sentiment in sarcastic or ambiguous tweets.
- Lexicons and rules need continuous updating.
- Slang, abbreviations and misspellings reduce accuracy.
- A single model can overfit or underfit on noisy social-media text.

## 2.3 Proposed System

The proposed system learns sentiment patterns from labelled tweets. The tweets are cleaned and preprocessed; Word-level and Character-level TF-IDF features are extracted and combined; a Random Forest classifier, which is an ensemble of many decision trees, is trained on the combined features; and a hybrid step adds rule-based analysis on top of the machine learning prediction. The final Positive, Negative or Neutral label is shown through a Flask web interface.

## 2.4 Software Requirements

- **Operating System:** Windows 10 / Linux / macOS
- **Programming Language:** Python
- **Libraries:** Pandas, NumPy, NLTK, Scikit-learn, Joblib, Flask
- **IDE:** VS Code
- **Front End:** HTML, CSS
- **Data Format:** CSV (Kaggle dataset file)

Each component in Table 2.1 has a clear role in the system. Python is used for all stages of the project, from data preparation to the web interface. Pandas and NumPy handle the loading and cleaning of the dataset and the numerical operations on the feature matrices. NLTK and regular expressions are used for the text-cleaning steps. Scikit-learn provides the TF-IDF vectorizers and the Random Forest classifier, and Joblib saves the trained model and the fitted vectorizers so that they do not have to be rebuilt every time the application starts.

Flask is a lightweight web framework that serves the single web page and passes the text entered by the user to the prediction module. The page itself is built with HTML and CSS, so no additional client-side framework is required. Because all of these tools are freely available and run on a standard personal computer, the system can be set up and tested without special hardware or paid software.

**Table 2.1:** Software Requirements

| Component | Technology |
|---|---|
| Programming language | Python |
| Data processing | Pandas, NumPy |
| Text processing | NLTK / regular expressions |
| Feature extraction | TF-IDF (Scikit-learn) |
| Machine learning | Scikit-learn; Random Forest model |
| Model saving | Joblib |
| Web interface | Flask |
| Front end | HTML, CSS |
| Operating system and IDE | Windows 10 / Linux / macOS; VS Code |

The software stack is portable: the same Python code runs on Windows, Linux and macOS, and the web interface works in any modern browser. The libraries are installed in a virtual environment, so the versions used by this project do not interfere with other Python projects on the same computer, and the project can be moved to another machine by installing the same libraries and copying the saved model files.

## 2.5 Hardware Requirements

- **Processor:** Intel Core i3 or i5
- **RAM:** 8 GB or higher
- **Storage:** 128 GB, SSD preferred
- **System Type:** 64-bit operating system

The hardware requirements in Section 2.5 are modest because the Random Forest is trained on a feature matrix of 61,121 rows and 15,000 columns, which fits comfortably in 8 GB of memory, and because a single prediction needs only the saved model and the two vectorizers. A solid-state drive is preferred only to shorten the time needed to load the saved model files.

## 2.6 Advantages of the Proposed System

- Learns sentiment patterns from labelled data instead of relying only on a word list.
- Character-level TF-IDF helps with spelling variations and informal text, while Word-level TF-IDF captures word and phrase patterns.
- Random Forest combines many decision trees by majority vote, which gives more reliable predictions than a single tree [9], [10].
- The hybrid rule-based layer adds a second source of judgement to the machine learning prediction.
- A simple web interface lets anyone test the model without programming knowledge.

## 2.7 Functional Requirements

**Table 2.2:** Functional Requirements

| ID | Requirement |
|---|---|
| FR1 | The system shall accept a tweet or sentence entered in a text box. |
| FR2 | The system shall provide an “Analyze Sentiment” button that starts the prediction. |
| FR3 | The system shall clean and preprocess the entered text in the same way as the training data. |
| FR4 | The system shall convert the text into combined Word and Character TF-IDF features and apply the trained Random Forest model. |
| FR5 | The system shall apply the hybrid rule-based analysis and produce the final sentiment: Positive, Negative or Neutral. |
| FR6 | The system shall display the predicted sentiment label in a prediction area. |
| FR7 | The system shall provide example inputs that the user can select and analyse. |

## 2.8 Non-Functional Requirements

- **Usability** – a single page with a clearly labelled text box, button and result area.
- **Performance** – the prediction should be returned quickly enough for interactive use.
- **Accuracy** – the model should achieve reliable classification on unseen text, as measured in Chapter 7.
- **Maintainability** – the preprocessing, feature extraction and model components are separate so that they can be retrained or replaced.
- **Portability** – the interface runs in a standard web browser.

# CHAPTER 3

# SYSTEM IMPLEMENTATION

## 3.1 Overview of Development Tools

### 3.1.1 Development Environment

The project is developed in Python using VS Code as the IDE and runs on Windows 10, Linux or macOS. Table 3.1 lists the technologies used in the project.

**Table 3.1:** Technologies Used

| Component | Technology |
|---|---|
| Programming language | Python |
| Data processing | Pandas, NumPy |
| Text processing | NLTK [17] / regular expressions |
| Feature extraction | TF-IDF (Scikit-learn) [16] |
| Machine learning | Scikit-learn |
| Model | Random Forest |
| Model saving | Joblib |
| Web interface | Flask [18] |
| Front end | HTML, CSS |
| Operating system, IDE | Windows 10 / Linux / macOS, VS Code |

### 3.1.2 Machine Learning Library (Scikit-learn)

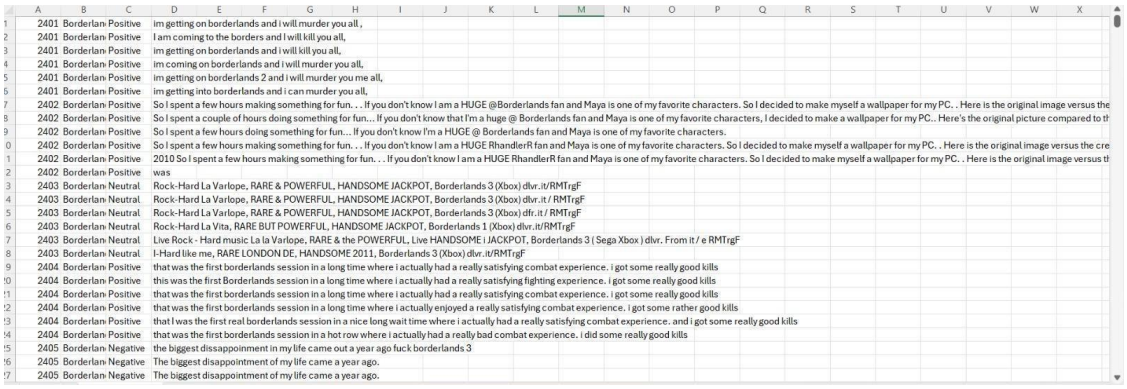
Scikit-learn [16] provides the Word-level and Character-level TF-IDF vectorizers and the Random Forest classifier used in this project. Joblib saves the trained model and the two fitted vectorizers in the models folder, so that they do not have to be rebuilt every time the application starts.

### 3.1.3 Web Framework (Flask)

Flask [18] is a lightweight web framework that serves the single web page and passes the text entered by the user to the prediction module. The page itself is built with HTML and CSS, so no additional client-side framework is required.

## 3.2 Dataset Description

The project uses the Twitter Entity Sentiment Analysis dataset from Kaggle, which contains 74,682 records. Each record has four fields: Tweet ID, Entity, Sentiment and Tweet. The sentiment classes used are Positive, Negative and Neutral. Fig 3.1 shows sample rows of the dataset.



| | A | B | C | D |
|---|---|---|---|---|
| 1 | 2401 | Borderlan | Positive | im getting on borderlands and i will murder you all , |
| 2 | 2401 | Borderlan | Positive | I am coming to the borders and I will kill you all, |
| 3 | 2401 | Borderlan | Positive | im getting on borderlands and i will kill you all, |
| 4 | 2401 | Borderlan | Positive | im coming on borderlands and i will murder you all, |
| 5 | 2401 | Borderlan | Positive | im getting on borderlands 2 and i will murder you me all, |
| 6 | 2401 | Borderlan | Positive | im getting into borderlands and i can murder you all, |
| 7 | 2402 | Borderlan | Positive | So I spent a few hours making something for fun. . . If you don't know I am a HUGE @Borderlands fan and Maya is one of my favorite characters. So I decided to make myself a wallpaper for my PC. . Here is the original image versus the |
| 8 | 2402 | Borderlan | Positive | So I spent a couple of hours doing something for fun... If you don't know that I'm a huge @ Borderlands fan and Maya is one of my favorite characters, I decided to make a wallpaper for my PC.. Here's the original picture compared to th |
| 9 | 2402 | Borderlan | Positive | So I spent a few hours doing something for fun... If you don't know I'm a HUGE @ Borderlands fan and Maya is one of my favorite characters. |
| 10 | 2402 | Borderlan | Positive | So I spent a few hours making something for fun. . . If you don't know I am a HUGE RhandlerR fan and Maya is one of my favorite characters. So I decided to make myself a wallpaper for my PC. . Here is the original image versus the cre |
| 11 | 2402 | Borderlan | Positive | 2010 So I spent a few hours making something for fun. . . If you don't know I am a HUGE RhandlerR fan and Maya is one of my favorite characters. So I decided to make myself a wallpaper for my PC. . Here is the original image versus th |
| 12 | 2402 | Borderlan | Positive | was |
| 13 | 2403 | Borderlan | Neutral | Rock-Hard La Varlope, RARE & POWERFUL, HANDSOME JACKPOT, Borderlands 3 (Xbox) dlvr.it/RMTrgF |
| 14 | 2403 | Borderlan | Neutral | Rock-Hard La Varlope, RARE & POWERFUL, HANDSOME JACKPOT, Borderlands 3 (Xbox) dlvr.it / RMTrgF |
| 15 | 2403 | Borderlan | Neutral | Rock-Hard La Varlope, RARE & POWERFUL, HANDSOME JACKPOT, Borderlands 3 (Xbox) dfr.it / RMTrgF |
| 16 | 2403 | Borderlan | Neutral | Rock-Hard La Vita, RARE BUT POWERFUL, HANDSOME JACKPOT, Borderlands 1 (Xbox) dlvr.it/RMTrgF |
| 17 | 2403 | Borderlan | Neutral | Live Rock - Hard music La la Varlope, RARE & the POWERFUL, Live HANDSOME i JACKPOT, Borderlands 3 ( Sega Xbox ) dlvr. From it / e RMTrgF |
| 18 | 2403 | Borderlan | Neutral | I-Hard like me, RARE LONDON DE, HANDSOME 2011, Borderlands 3 (Xbox) dlvr.it/RMTrgF |
| 19 | 2404 | Borderlan | Positive | that was the first borderlands session in a long time where i actually had a really satisfying combat experience. i got some really good kills |
| 20 | 2404 | Borderlan | Positive | this was the first Borderlands session in a long time where i actually had a really satisfying fighting experience. i got some really good kills |
| 21 | 2404 | Borderlan | Positive | that was the first borderlands session in a long time where i actually had a really satisfying combat experience. i got some really good kills |
| 22 | 2404 | Borderlan | Positive | that was the first borderlands session in a long time where i actually enjoyed a really satisfying combat experience. i got some rather good kills |
| 23 | 2404 | Borderlan | Positive | that I was the first real borderlands session in a nice long wait time where i actually had a really satisfying combat experience. and i got some really good kills |
| 24 | 2404 | Borderlan | Positive | that was the first borderlands session in a hot row where i actually had a really bad combat experience. i did some really good kills |
| 25 | 2405 | Borderlan | Negative | the biggest dissappoinment in my life came out a year ago fuck borderlands 3 |
| 26 | 2405 | Borderlan | Negative | The biggest disappointment of my life came a year ago. |
| 27 | 2405 | Borderlan | Negative | The biggest disappointment of my life came a year ago. |

**Fig 3.1:** Sample Rows of the Dataset (Tweet ID, Entity, Sentiment, Tweet)

The following observations are made only from the rows visible in Fig 3.1 (identifiers 2401 to 2405) and should be verified on the complete dataset:

- Several consecutive rows share the same identifier and carry slightly different wordings of what appears to be the same text.
- Some labels look questionable for the text shown, for example “i will murder you all” and a sentence describing a “really bad combat experience” are both labelled Positive, and a row consisting of the single word “was” is labelled Positive.
- Some rows labelled Neutral are product-listing style texts with garbled wording. **Table 3.2:** Dataset Summary

| Item | Value |
|---|---|
| Dataset | Twitter Entity Sentiment Analysis |
| Source | Kaggle |
| Records | 74,682 |
| Fields | Tweet ID, Entity, Sentiment, Tweet |
| Sentiment classes | Positive, Negative, Neutral |
| Records after cleaning | 61,121 |
| Class distribution | Negative 22,358; Positive 20,655; Neutral 18,108 |

## 3.3 Implementation Details

### 3.3.1 Project Files

The project is organised into the main files and folders listed in Table 3.3.

**Table 3.3:** Main Project Files

| File / folder | Purpose |
|---|---|
| app.py | Flask application that serves the web page and handles requests |
| predict.py | Loads the saved model and vectorizers and predicts the sentiment of new text |
| preprocess.py | Text cleaning and preprocessing steps |

| | |
|---|---|
| train_model.py | Feature extraction, Random Forest training and saving of the model |
| models/ | Saved model and vectorizers (Joblib) |
| templates/ | HTML templates of the web interface |

### 3.3.2 Dataset Loading

The dataset is read into a Pandas data frame with the fields Tweet ID, Entity, Sentiment and Tweet (Fig 3.1). The training script reports the original number of rows (74,682), the number of rows after filtering (61,121) and the sentiment distribution. Records without tweet text are removed first (Table 5.1).

### 3.3.3 Preprocessing Implementation

The preprocessing steps of Table 5.1 are implemented with regular expressions (and NLTK where used) in a single function that is applied to every tweet during training and to every text entered by the user, so that the model always receives text in the same form.

### 3.3.4 Feature Extraction Implementation

A Word-level TF-IDF vectorizer (10,000 features) and a Character-level TF-IDF vectorizer (5,000 features) are fitted on the training text, and their outputs are combined into one feature set of 15,000 features (Section 5.5). Both fitted vectorizers are saved with Joblib.

### 3.3.5 Random Forest Implementation

The Random Forest classifier from Scikit-learn is trained on the combined features of the training set. Its parameters are listed in Table 6.1. The trained model is saved with Joblib in the models folder.

### 3.3.6 Hybrid Rule-Based Layer

After the Random Forest has predicted a class, the rule-based analysis is applied to the prediction to produce the final sentiment (Fig 4.5). This layer is implemented in the prediction module and gives a second source of judgement on top of the machine learning prediction.

### 3.3.7 Prediction Module

When the user presses the Analyze Sentiment button, the Flask application passes the entered text to the prediction module. The module preprocesses the text, converts it with the saved vectorizers, applies the saved Random Forest model and the rule-based analysis, and returns Positive, Negative or Neutral, which is shown in the Prediction area of the page (Figs 8.2 and 8.3).

The prediction steps are carried out in the following order for every text entered by the user:

- The Flask application receives the text from the form on the web page.
- The text is cleaned with the same preprocessing function that was used for the training data.
- The saved Word-level and Character-level vectorizers convert the cleaned text into a combined feature vector of 15,000 values.
- The saved Random Forest model predicts the class by majority vote of its 100 trees.
- The rule-based analysis confirms or adjusts the prediction to give the final label.
- The label (Positive, Negative or Neutral) is returned to the page and shown in the Prediction card.

### 3.3.8 User Interface

The user interface is a Flask web page built with HTML and CSS, shown in Fig 8.1. The elements visible in the screenshot are listed below.

- A heading, “Sentiment Analysis”, with the sub-heading “Using Ensemble Learning”.
- A multi-line text box with the placeholder text “Enter a tweet or sentence...”.
- An “Analyze Sentiment” button that runs the prediction.
- An “Example Inputs” list with three ready-made sentences: “I really love this game”, “I hate this product” and “The game is okay”, each marked with an emoticon.
- A “Prediction” card that appears after analysis and shows the predicted label (Figs 8.2 and 8.3). The card shows only the class label; no probability or confidence value is displayed.

### 3.3.9 Source Code

The important parts of the source code, with explanations, are given in Appendix A.

The source code is divided into four main parts, which correspond to the files listed in Table 3.3. The preprocessing file contains the text-cleaning function of Table 5.1. The training file loads the dataset, applies the cleaning function, builds the Word-level and Character-level TF-IDF features, splits the data into 80% training and 20% testing samples, trains the Random Forest and prints the accuracy and the classification report (Fig 6.1). It also saves the confusion matrix, the model and the two vectorizers. The prediction file loads these saved objects and returns the final sentiment for a given text, and the application file connects the prediction module to the web page.

Keeping the preprocessing, training, prediction and interface code in separate files makes the project easier to maintain: the cleaning rules, the feature settings or the model can be changed in one place and the system retrained without modifying the web interface.

# CHAPTER 4
# SYSTEM ARCHITECTURE

## 4.1 System Architecture

The system consists of a Flask web interface, a prediction module and a saved model. The user enters text in the browser; the prediction module loads the model and vectorizers saved with Joblib, preprocesses the text, converts it into Word and Character TF-IDF features, applies the Random Forest classifier and then the rule-based analysis; and the final label is returned to the interface and displayed.

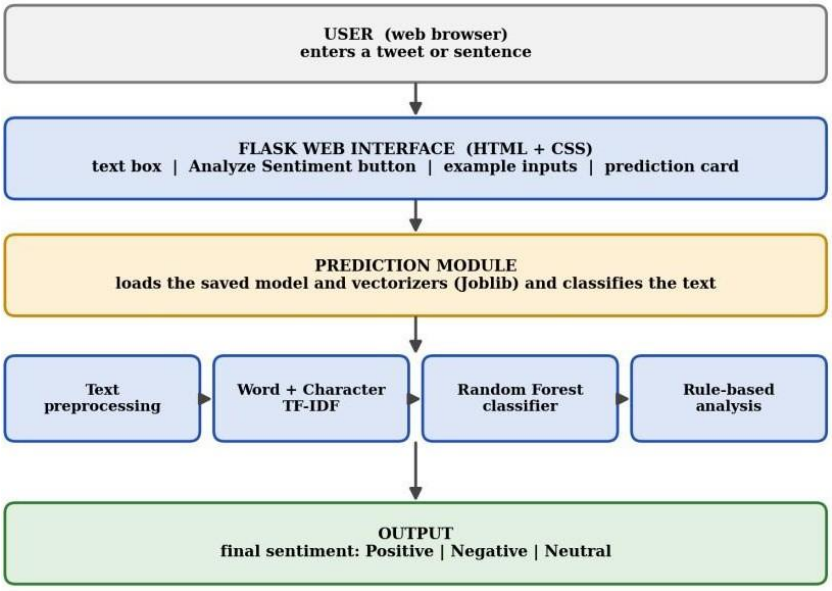



**Fig 4.1:** System Architecture of the Sentiment Analysis System

## 4.2 Workflow of the System

The workflow has eleven steps in four groups, as shown in Fig 4.2: data (steps 1 to 3), features (steps 4 to 6), model (steps 7 to 9) and output (steps 10 and 11). The first eight steps build the model; the prediction, hybrid analysis and output steps are repeated for every text entered by a user.

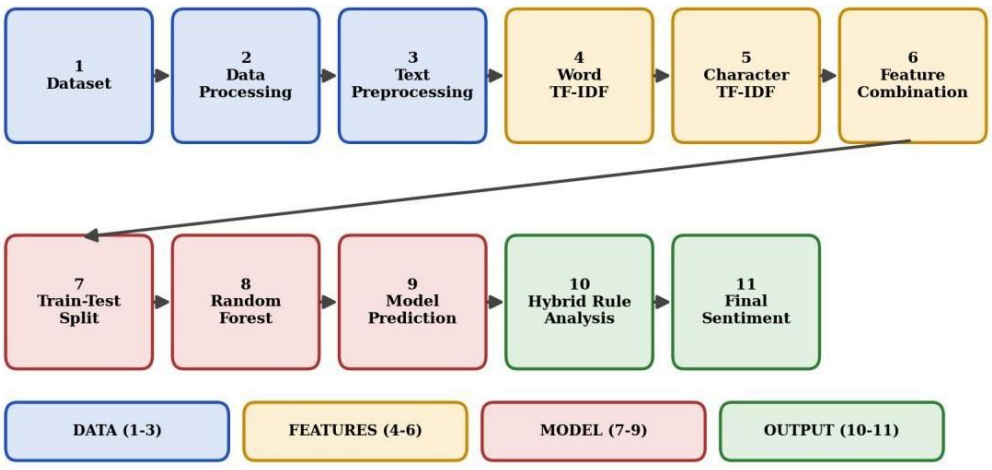



**Fig 4.2:** Workflow of the Sentiment Analysis System

## 4.3 Data Flow Diagram

Fig 4.3 is the level-0 data flow diagram. The user supplies text and receives a predicted sentiment; the system loads the model and vectorizers that were saved after training.

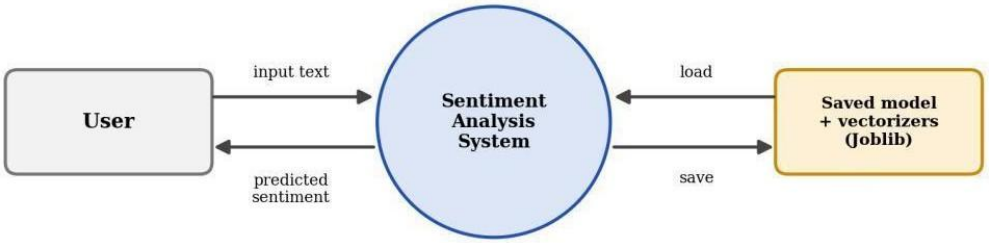



**Fig 4.3:** Level-0 Data Flow Diagram

## 4.4 Dataset Processing Pipeline

Each tweet passes through the seven processing steps shown in Fig 4.4 before it is converted into TF-IDF features. The same steps are applied to the training data and to every text entered at prediction time.

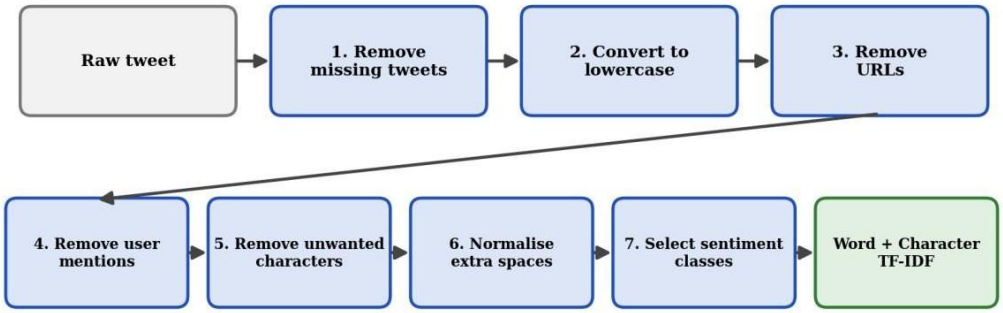



**Fig 4.4:** Data Processing Pipeline

## 4.5 Machine Learning Model and Ensemble Architecture

The combined Word and Character TF-IDF vector of a text is given to a Random Forest, which consists of B decision trees (B = 100 in this project). Each tree predicts a class and the class chosen by most trees is the model prediction. The hybrid rule-based analysis is then applied to this prediction to give the final sentiment (Fig 4.5).

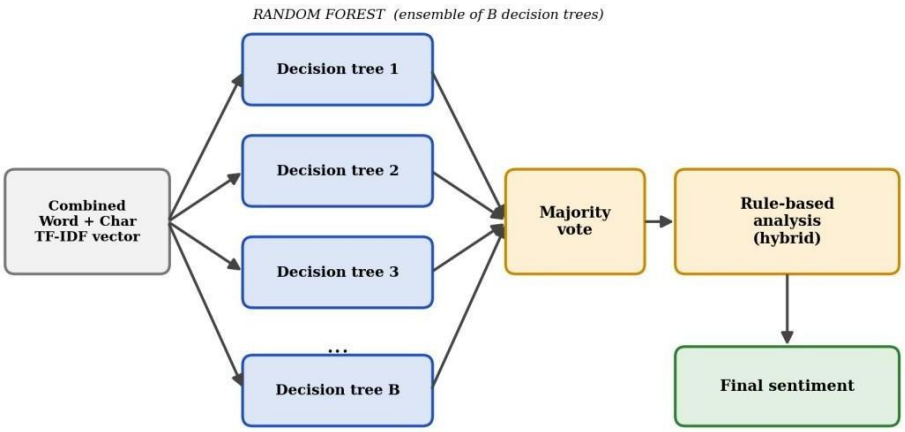



**Fig 4.5:** Random Forest and Hybrid Ensemble Architecture

## 4.6 Inputs and Outputs of the Stages

**Table 4.1:** Inputs and Outputs at Each Stage

| Stage | Input | Output |
|---|---|---|
| Dataset | Kaggle Twitter Entity Sentiment Analysis file | 74,682 labelled tweet records |
| Data processing and text preprocessing | Raw tweets | 61,121 cleaned, lower-case tweets in the three selected sentiment classes |
| Word and Character TF-IDF | Cleaned text | Word features (10,000) and character features (5,000), combined into 15,000 features |
| Train–test split | Combined features with labels | Training set (48,896) and test set (12,225) |
| **Stage** | **Input** | **Output** |
| Random Forest training | Training set | Trained Random Forest model, saved with the vectorizers |
| Model prediction | Features of a text | Class chosen by majority vote of the trees |
| Hybrid rule analysis | Model prediction and the text | Final sentiment: Positive, Negative or Neutral |
| Web output | Text entered by the user | Predicted sentiment label |

Table 4.1 shows that every stage has a clearly defined input and output, so the stages can be developed and tested one at a time. The first four stages build the training data: the raw file is reduced from 74,682 to 61,121 cleaned tweets, converted into 15,000 numerical features per tweet and divided into 48,896 training and 12,225 test samples. The Random Forest training stage then produces the saved model, and the remaining stages are repeated for every text entered by a user.

An important design decision is that the preprocessing and the two fitted vectorizers are saved and reused at prediction time. This guarantees that a new tweet is transformed in exactly the same way as the training data, which is a basic requirement for a trained model to give meaningful predictions on unseen text.

# CHAPTER 5

# DATA PREPROCESSING

## 5.1 Introduction

Preprocessing converts the raw tweets into clean text and then into numerical features that the classifier can use. Pandas and NumPy handle the loading and cleaning of the dataset and the numerical operations on the feature matrices, NLTK and regular expressions are used for the text-cleaning steps, and Scikit-learn provides the TFIDF vectorizers. The same preprocessing is applied to the training data and to every text entered by the user, so that the model always receives text in the same form.

## 5.2 Handling Missing Values

In the first processing step, every record that has no tweet text is removed, so that only records with text are passed to the later steps (Table 5.1). Together with the selection of the three sentiment classes in step 7, the cleaning reduces the dataset from 74,682 to 61,121 records.

## 5.3 Data Cleaning

The presentation documents seven processing steps, which are applied in order. Table 5.1 lists them with the example tweet used in the presentation, showing the text after each step, and Fig 5.1 shows the same steps as a diagram.

**Table 5.1:** Data Processing Steps with an Example Tweet

| Step | Processing step | Tweet after the step |
|---|---|---|
| – | Input tweet | I LOVE this GAME!! @Xbox https://t.co/a1b2 |
| 1 | Remove missing tweets | (rows without tweet text are dropped) |
| 2 | Convert to lowercase | i love this game!! @xbox https://t.co/a1b2 |
| 3 | Remove URLs | i love this game!! @xbox |
| 4 | Remove user mentions | i love this game!! |
| 5 | Remove unwanted characters | i love this game |
| 6 | Normalise extra spaces | i love this game |
| 7 | Select sentiment classes | i love this game → Label: Positive (kept) |

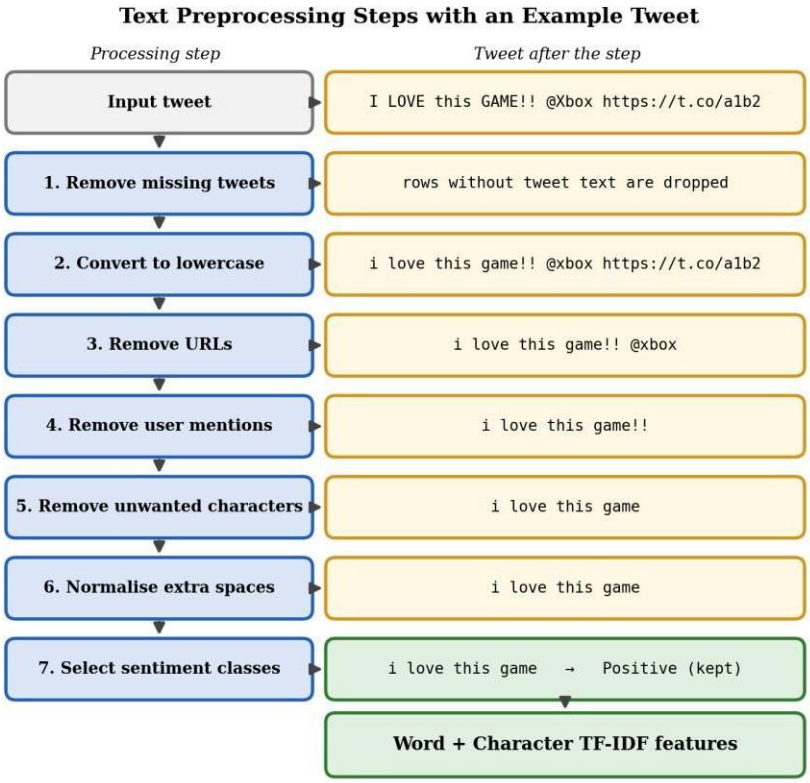



**Fig 5.1:** Text Preprocessing Steps with an Example Tweet

Removal of missing tweets, lowercasing, removal of URLs and mentions, removal of unwanted characters and normalisation of spaces make the text uniform, so that the same word is represented in the same form wherever it occurs. Step 7 keeps the three sentiment classes used by the project. The number of records removed in steps 1 and 7 is 13,561, which reduces the dataset from 74,682 to 61,121 records (74,682 − 13,561 = 61,121), as summarised in Fig 5.2. Stop-word removal, stemming and lemmatisation are not among the documented steps; because stop-word removal is not applied, negation words such as “not” remain in the text passed to the vectorizers.




**Fig 5.2:** Dataset Cleaning Summary

Because the sample in Fig 3.1 contains near-duplicate rows, the handling of duplicates deserves attention: if nearidentical texts are present in both the training and test sets, the measured accuracy may appear higher than the real accuracy. Duplicate removal is not one of the implemented steps and is listed as a future enhancement (Chapter 10).

## 5.4 Outlier Detection and Range Validation

The tweets are text, so numerical outlier-detection methods such as boxplots, the IQR rule or z-scores do not apply to this dataset. Validity of the records is ensured instead by step 7 of Table 5.1, which keeps only the records whose sentiment label is one of the three classes used by the project: Positive, Negative and Neutral. Unusual spellings and elongated words are not removed; they are handled by the Character-level TF-IDF features described in Section 5.5.

## 5.5 Feature Engineering / Encoding

A classifier needs numbers rather than text. The project uses TF-IDF, in which each term receives a weight that grows with its frequency in a document and shrinks with the number of documents containing it [6]. For a term t in a document d from a collection of N documents:

$$TF\text{-}IDF(t, d) = tf(t, d) \times \log( N / df(t) ) \qquad (5.1)$$

where tf(t, d) is the number of times t occurs in d and df(t) is the number of documents containing t. Two TF-IDF representations are computed from the same cleaned text and combined into one feature set (Fig 5.3).

**Table 5.2:** Word-Level and Character-Level TF-IDF

| Feature type | What it captures | Benefit for tweets |
|---|---|---|
| Word TF-IDF (10,000 features) | Word and phrase patterns; gives more weight to important, less common words | Identifies sentiment-bearing words and phrases |
| Character TF-IDF (5,000 features) | Character patterns inside words | Helps with spelling variations and informal text |

The word vectorizer produces a matrix of shape (61,121 × 10,000) and the character vectorizer a matrix of shape (61,121 × 5,000); combined, the final feature matrix has 15,000 columns. The fitted vectorizers are saved with Joblib so that the web interface applies the identical transformation to new text.




**Fig 5.3:** Combination of Word-Level and Character-Level TF-IDF Features

## 5.6 Data Visualization

After cleaning, the three classes are reasonably balanced: Negative is the largest class (36.6%) and Neutral the smallest (29.6%). Fig 5.4 shows the class distribution of the cleaned dataset.

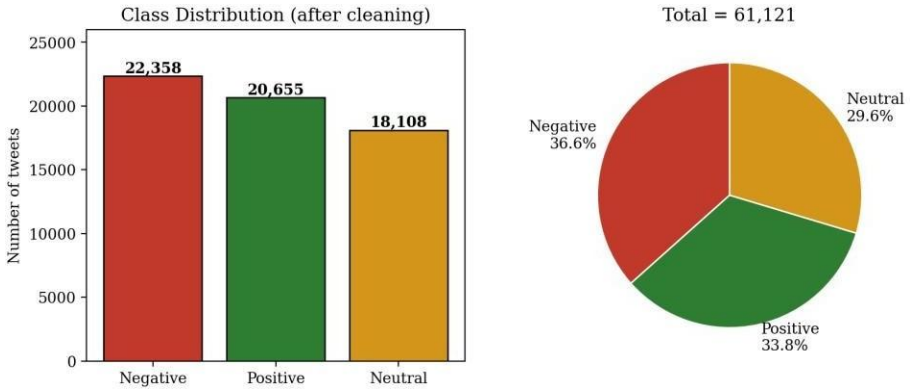



**Fig 5.4:** Class Distribution of the Cleaned Dataset

## 5.7 Train-Test Split

The processed data is divided into a training set, used to fit the Random Forest, and a test set held out to measure performance on unseen text. The split is 80% for training and 20% for testing: of the 61,121 cleaned records, 48,896 are used for training and 12,225 for testing (Fig 5.5). Because the dataset contains near-duplicate texts under the same identifier, the splitting procedure should ensure that such rows do not appear in both sets. After training, the model and the vectorizers are saved with Joblib and loaded by the prediction module for new text.

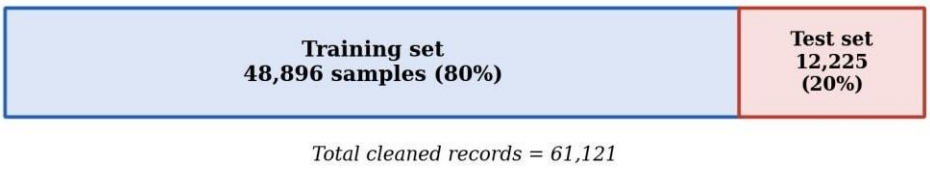



**Fig 5.5:** Train–Test Split of the Cleaned Dataset

## 5.8 Summary

The dataset has 74,682 records with the classes Positive, Negative and Neutral (Table 3.2). After cleaning, 61,121 records remain (Negative 22,358; Positive 20,655; Neutral 18,108). Of these, 48,896 records are used for training and 12,225 for testing. Fig 5.6 shows how the number of records changes through the pipeline.




**Fig 5.6:** Number of Records at Each Stage of the Pipeline

# CHAPTER 6
# METHODOLOGY AND ALGORITHMS

## 6.1 Introduction

The project applies supervised classification. Each training example is a tweet with a known sentiment label. The tweets are cleaned, converted into numerical features using Word-level and Character-level TF-IDF, and used to train a Random Forest classifier. A hybrid rule-based analysis is applied to the model prediction to give the final sentiment. General descriptions and formulas in this chapter follow standard machine learning theory; the settings of the models used in the project are listed in Table 6.1.

## 6.2 Classification Techniques

### 6.2.1 Decision Tree

**Working:** A decision tree repeatedly splits the data on feature values, choosing at each node the split that makes the resulting groups purer. One common impurity measure is the Gini impurity, where pk is the proportion of class k at a node:

$$G = 1 - \Sigma k\, pk^2 \qquad (6.1)$$

**Advantages:** A decision tree is simple to understand and needs no feature scaling. Its main weakness is that a single tree may overfit the training data, which is addressed by combining many trees in a Random Forest.

**Application in Project:** Decision trees are the base learners of the Random Forest used in this project; each of the 100 trees is trained on a bootstrap sample of the training data and uses a random subset of the features at each split.

### 6.2.2 Random Forest

**Working:** Random Forest is an ensemble learning method that combines many decision trees [10]. A Random Forest builds B trees, each trained on a bootstrap sample of the training data and using a random subset of features at each split, which makes the trees diverse [9]. Each tree predicts a class, and the final class is chosen by majority vote:

$$\hat{y} = mode\ \{\ h1(x),\ h2(x),\ \ldots,\ hB(x)\ \} \qquad (6.2)$$

**Advantages:** Individual trees may overfit the training data, but because they are trained on different samples and feature subsets, their errors are partly uncorrelated, and aggregating them reduces variance and generally improves generalisation. This also addresses the limitation of a single model that can overfit or underfit on noisy text.

**Application in Project:** A Random Forest of 100 trees is trained on the combined 15,000 Word and Character TF-IDF features of the 48,896 training tweets. The settings used are listed in Table 6.1.

**Table 6.1:** Random Forest Settings

| Setting | Value |
|---|---|
| Number of trees | 100 |
| Maximum depth | None |

| | |
|---|---|
| Class weights / other parameters | class_weight = 'balanced' |
| Random seed | 42 |

## 6.3 Ensemble and Hybrid Technique

### 6.3.1 Ensemble Learning (Random Forest)

**Working:** The ensemble technique used in the project is the Random Forest, a bagging-type ensemble of decision trees whose predictions are combined by majority vote [9], [10] (Section 6.2.2).

**Advantages:** Combining many trees reduces variance, keeps training inexpensive and keeps the results easy to explain.

### 6.3.2 Hybrid Rule-Based Technique

On top of this, a hybrid approach applies rule-based analysis to the machine learning prediction to give the final sentiment. Table 6.2 summarises the two layers.

**Table 6.2:** Layers of the Hybrid Ensemble Approach

| Layer | Method | Role |
|---|---|---|
| Machine learning layer | Random Forest on combined Word and Character TF-IDF features | Learns sentiment patterns from labelled tweets; the trees vote and the majority class becomes the prediction |
| Rule-based layer | Rule-based analysis (hybrid step 10 of the workflow) | Adjusts or confirms the model prediction to give the final sentiment |

The specific rules and the way the rule result is combined with the model prediction are given in the source code (Appendix A). It should be noted that the ensemble here is formed within a single algorithm; the project does not combine different types of classifiers by voting or stacking [8], [12], [13], which is mentioned as a possible enhancement in Chapter 10.

## 6.4 Model Training and Output

The Random Forest is trained on the 48,896 training samples (Section 5.7). After training, the model and the two vectorizers are saved with Joblib and loaded by the prediction module for new text.

Fig 6.1 shows the complete output of the training script, including the dataset summary, the feature shapes, the split sizes and the model performance. The Random Forest achieved an accuracy of 90.94% on the 12,225 heldout test tweets.

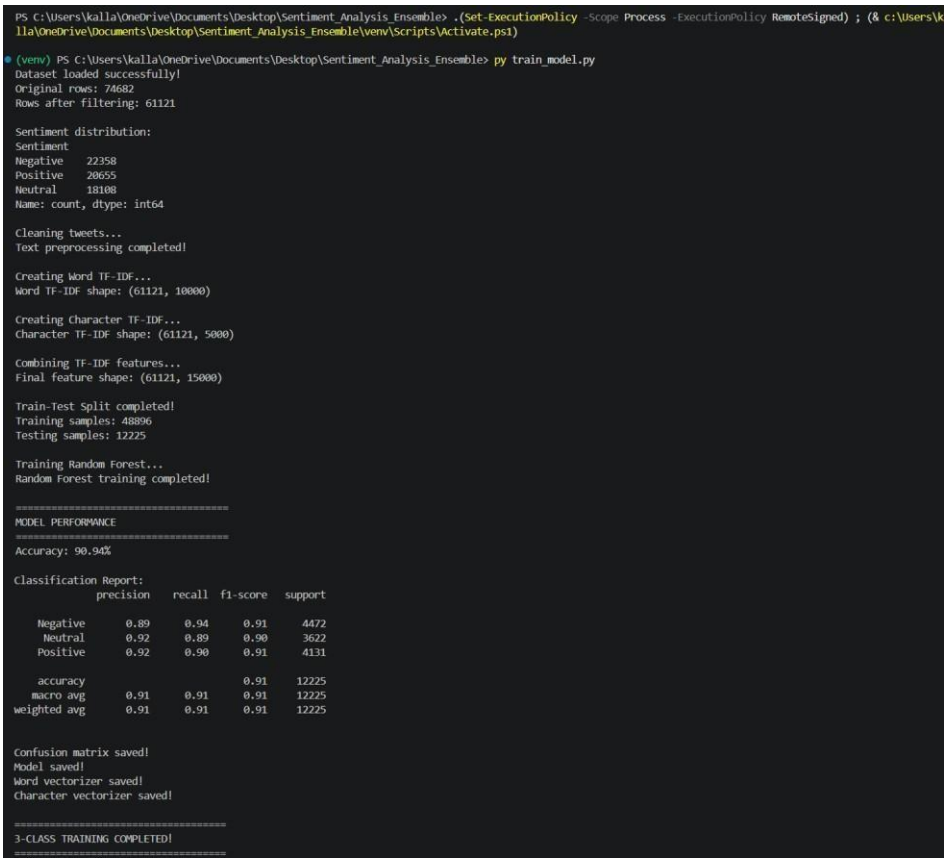



**Fig 6.1:** Output Screen of the Training Script (train_model.py)

## 6.5 Model Evaluation Metrics

Sentiment classification here has three classes, so the confusion matrix is a 3 × 3 table in which each row is the true class and each column is the predicted class. For a class treated as positive, the terms TP, TN, FP and FN have their usual meaning, and the following metrics are computed:

$$Accuracy = (TP + TN) / (TP + TN + FP + FN) \qquad (6.3)$$

$$Precision = TP / (TP + FP) \qquad (6.4)$$

$$Recall = TP / (TP + FN) \qquad (6.5)$$

$$F1 = 2 \times Precision \times Recall / (Precision + Recall) \qquad (6.6)$$

- Accuracy — overall correctness of predictions
- Precision — correctness of positive predictions
- Recall — ability to find all relevant instances
- F1-Score — harmonic mean of precision and recall

Accuracy is the proportion of correctly classified tweets. Precision is the proportion of tweets predicted as a class that truly belong to it. Recall is the proportion of tweets of a class that are found. F1-score is the harmonic mean of precision and recall. Per-class values are averaged (macro average and weighted average) to give a single figure. When the classes are not balanced, accuracy should be read together with the other metrics.

In this project the metrics are interpreted class by class. A high recall for the Negative class means that few negative tweets are missed, which matters when negative messages need attention first, while a high precision for the Positive and Neutral classes means that tweets labelled with these classes are rarely wrong. The macro average treats the three classes equally, whereas the weighted average takes into account how many test tweets belong to each class; because the classes are fairly balanced, the two averages are almost the same (Chapter 7).

# CHAPTER 7
# RESULTS AND ANALYSIS

## 7.1 Introduction

This chapter reports the results of the Random Forest model and the outputs of the web application. The numerical results are taken from the output of the training script (Fig 6.1), and the sentence-level outputs come from the project presentation and the supplied screenshots.

## 7.2 Experimental Setup

**Table 7.1:** Experimental Setup

| Item | Details |
|---|---|
| Task | Classification of tweets into Positive, Negative and Neutral sentiment |
| Features | Combined Word-level and Character-level TF-IDF (15,000 features) |
| Model | Random Forest (ensemble of decision trees), followed by hybrid rule-based analysis |
| Train / test split | 80% training (48,896 samples) / 20% testing (12,225 samples) |
| Metrics | Accuracy, precision, recall, F1-score, confusion matrix |

## 7.3 Random Forest Results

### 7.3.1 Classification Report

**Table 7.2:** Per-Class Classification Report

| Class | Precision | Recall | F1-Score | Support |
|---|---|---|---|---|
| Positive | 0.92 | 0.90 | 0.91 | 4,131 |
| Negative | 0.89 | 0.94 | 0.91 | 4,472 |
| Neutral | 0.92 | 0.89 | 0.90 | 3,622 |

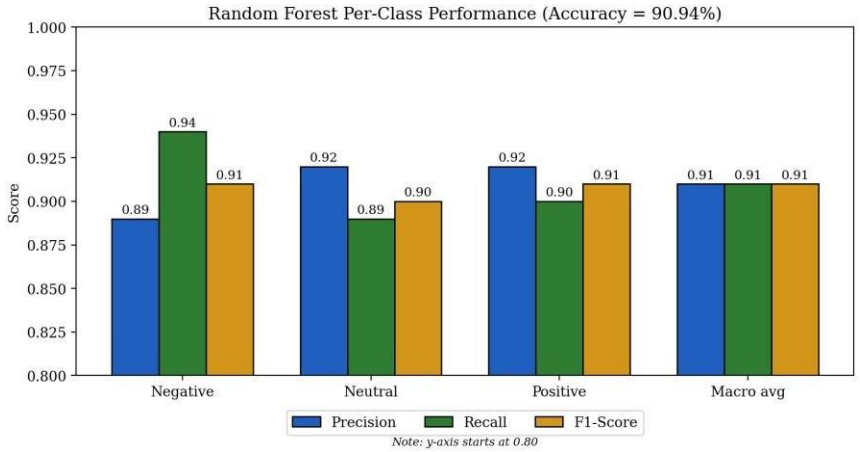



**Fig 7.1:** Per-Class Precision, Recall and F1-Score of the Random Forest

The three classes are classified at a similar level, with F1-scores between 0.90 and 0.91, and the macro and weighted averages of precision, recall and F1-score are all 0.91. The Negative class has the highest recall (0.94) but the lowest precision (0.89): the model finds most negative tweets, but some tweets of the other classes are also labelled Negative. The Neutral class has the lowest recall (0.89), which is a common difficulty because neutral text lies between the two polar classes.

### 7.3.2 Confusion Matrix

The confusion matrix of the Random Forest model on the test set is shown in Fig 7.2 and Table 7.3. In the figure, each row is the true class and each column is the predicted class; the diagonal cells are correct predictions, and the off-diagonal cells show which classes are confused with each other.

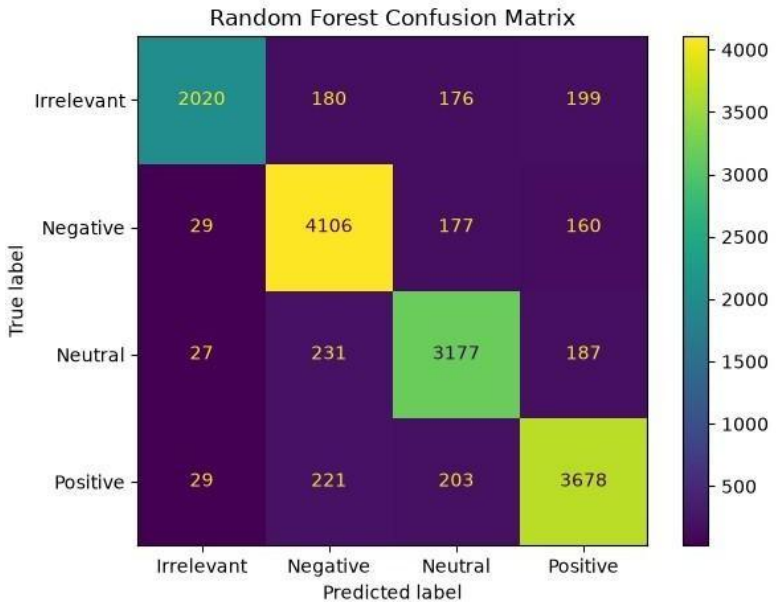



**Fig 7.2:** Confusion Matrix of the Random Forest Model

**Table 7.3:** Confusion Matrix (rows: actual, columns: predicted)

| Actual \ Predicted | Positive | Negative | Neutral |
|---|---|---|---|
| Positive | | | |
| Negative | | | |
| Neutral | | | |

The diagonal cells are the correctly classified tweets, and the off-diagonal cells show the most frequent confusions between the classes. The row totals correspond to the support values of Table 7.2.

## 7.4 Hybrid Rule-Based System Results

The system was tested with new, unseen text. Table 7.4 gives the three sentences reported in the presentation, and Figs 8.2 and 8.3 show the web application classifying two further sentences. These outputs are demonstrations and not a statistical evaluation.

**Table 7.4:** Predictions for Test Sentences (from the project presentation)

| Input | Predicted sentiment |
|---|---|
| I really love this game | Positive |
| I hate this product | Negative |
| The game is okay | Neutral |

## 7.5 Comparison of Models

**Table 7.5:** Performance of the Random Forest Model on the Test Set

| Model | Accuracy | Precision | Recall | F1-Score |
|---|---|---|---|---|
| Random Forest | 90.94% | 0.91 | 0.91 | 0.91 |
| Random Forest + rule-based analysis (final system) | Not measured separately | – | – | – |

The precision, recall and F1-score in the first row are macro averages over the three classes. The second row was not measured separately, so the measured accuracy refers to the Random Forest model on its own.

## 7.6 Key Findings

- The system classifies clearly positive, clearly negative and neutral example sentences as expected (Table 7.4), and gives a plausible label for a mixed-sentiment sentence (Fig 8.3).
- The Random Forest reaches an accuracy of 90.94% on unseen test tweets, with similar performance on all three classes.
- Combining Word and Character TF-IDF is intended to handle slang and spelling variation; whether it improves accuracy over word features alone was not tested.
- The model is a single Random Forest followed by rule-based analysis. No other classifiers were trained, so the report cannot state that this ensemble is better than other models; such a claim would need a comparison.
- The sample rows of the dataset show near-duplicate texts and some labels that appear inconsistent with the text (Section 3.2). If this is also true of the full dataset, the labels contain noise, which can limit the accuracy any model can reach and can make test-set accuracy optimistic if similar rows fall into both the training and test sets.

## 7.7 Testing

### 7.7.1 Test Plan

Testing checks that the system behaves as specified in the functional requirements (Table 2.2). Test cases are grouped into sentiment prediction, interface, preprocessing, ensemble and error-handling tests, and each records the input, the expected outcome and the actual outcome. Three statuses are used so that expected outcomes are never presented as results: **Verified** means the actual result is shown in a screenshot in this report; **Reported**

means the actual result was observed and stated in the project presentation; **Pending** means no actual result has been recorded yet.

The tests were carried out on the web application running locally: the Flask server was started on a personal computer and the page was opened in a standard web browser. The same saved model and vectorizers that are used in normal operation were loaded, so the results reflect the behaviour of the complete system and not only of the classifier.

### 7.7.2 Sentiment Prediction Test Cases

**Table 7.6:** Sentiment Prediction Test Cases

| ID | Input text | Expected | Actual | Status |
|---|---|---|---|---|
| TC1 | i love this product | Positive | Positive (Fig 8.2) | Verified |
| TC2 | product is good but not working | Negative (main complaint) | Negative (Fig 8.3) | Verified |
| TC3 | I really love this game | Positive | Positive | Reported |
| TC4 | I hate this product | Negative | Negative | Reported |
| TC5 | The game is okay | Neutral | Neutral | Reported |

The expected result of TC2 is an assumption made for the test plan: the sentence mixes positive and negative sentiment, and Negative is accepted because the sentence ends with the problem. A different reader could label it differently.

The five sentiment test cases were chosen to cover the main situations that the system must handle: a clearly positive sentence (TC1, TC3), a clearly negative sentence (TC4), a neutral sentence (TC5) and a sentence with mixed sentiment (TC2). All five produced the expected label. The mixed sentence is the most informative case, because it contains the word “not”; since stop words are not removed during preprocessing, this negation remains in the text that reaches the vectorizers and can influence the prediction.

### 7.7.3 Interface Test Cases

**Table 7.7:** User Interface Test Cases

| ID | Test | Expected | Actual | Status |
|---|---|---|---|---|
| TC6 | Open the page | Heading, text box, button and example inputs are shown | As shown in Fig 8.1 | Verified |
| TC7 | Enter text and press Analyze Sentiment | A Prediction card with a label appears | As shown in Figs 8.2 and 8.3 | Verified |
| TC8 | Click an example input | Example is analysed or placed in the text box | – | Pending |

The interface tests check that the page contains the elements required by the functional requirements (FR1, FR2, FR6 and FR7) and that a prediction card appears after the Analyze Sentiment button is pressed. The example

inputs offered on the page are the same three sentences that are used in the sentiment test cases, so they can be used to repeat the basic tests quickly.

### 7.7.4 Text Preprocessing Tests

**Table 7.8:** Preprocessing Test Cases

| ID | Input | Expected | Actual | Status |
|---|---|---|---|---|
| TC9 | I LOVE this GAME!! @Xbox https://t.co/a1b2 | i love this game | i love this game (Table 5.1) | Reported |
| TC10 | Tweet with only a URL or mention | Empty text handled without failure | – | Pending |
| TC11 | Sentence with negation (“not good”) | Negation preserved in the cleaned text | – | Pending |

Preprocessing is tested separately because the same cleaning function is applied to the training data and to every text entered by a user (FR3). TC9 confirms that the example tweet is cleaned exactly as shown in Table 5.1. TC10 and TC11 cover two special situations: a tweet that becomes empty after cleaning, and a sentence with negation, whose words must be preserved.

### 7.7.5 Ensemble Model Tests

**Table 7.9:** Ensemble Model Test Cases

| ID | Test | Expected | Actual | Status |
|---|---|---|---|---|
| TC12 | Compare the Random Forest output with the majority vote of its trees on the same text | Both give the same class | – | Pending |
| TC13 | Compare the final sentiment with the Random Forest output alone on sample texts | Differences occur only where a rule applies | – | Pending |
| TC14 | Load the saved model and vectorizers in a fresh session | Same predictions as before saving | – | Pending |

The ensemble tests check that the final result is consistent with the way the Random Forest works: the class returned by the forest must be the class chosen by the majority of its trees (TC12), the rule-based layer must change the prediction only when a rule applies (TC13), and a model loaded from the saved files must give the same predictions as the model that was trained (TC14).

### 7.7.6 Error-Handling Tests

**Table 7.10:** Error-Handling Test Cases

| ID | Input | Expected | Actual | Status |
|---|---|---|---|---|
| TC15 | Empty text box, press Analyze Sentiment | No crash; a message or no prediction | – | Pending |

| | | | | |
|---|---|---|---|---|
| TC16 | Very long text | Handled without failure | – | Pending |
| TC17 | Text containing only symbols or digits | Handled without failure | – | Pending |

The error-handling tests check that the application stays stable when the input is not a normal sentence, such as an empty text box, a very long text or a text that contains only symbols or digits. These cases do not test the accuracy of the model; they make sure that the interface does not fail and that the user always receives either a label or a clear message.

### 7.7.7 Summary of Testing

Four test cases (TC1, TC2, TC6, TC7) are verified by screenshots, and four more (TC3, TC4, TC5, TC9) are reported in the project presentation; all of these produced the expected outcome. The remaining cases are defined but have no recorded result and should be executed and updated before the report is submitted.

- Seventeen test cases are defined in total: five sentiment prediction tests, three interface tests, three preprocessing tests, three ensemble tests and three error-handling tests.
- Eight cases have a recorded result (four verified by screenshots and four reported), and all eight gave the expected outcome.
- Nine cases (TC8, TC10 to TC17) are still pending, mainly edge cases and consistency checks of the ensemble, and they should be run to complete the test record.

# CHAPTER 8
# OUTPUT SCREENS

## 8.1 Web Interface

Fig 8.1 shows the web interface of the system, in which the user enters a tweet or sentence and presses the Analyze Sentiment button.

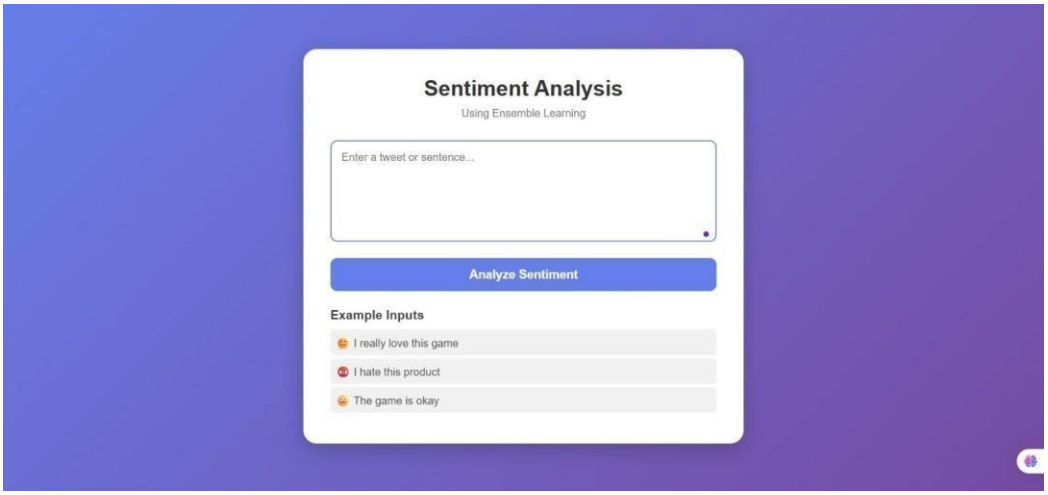



**Fig 8.1:** Web Interface of the Sentiment Analysis System

## 8.2 Sample Prediction Outputs




**Fig 8.2:** Prediction Output for "i love this product" (Positive)




**Fig 8.3:** Prediction Output for "product is good but not working" (Negative)

The sentence “i love this product” is predicted as Positive. The sentence “product is good but not working” contains both a positive phrase and a negative phrase and is predicted as Negative, which is consistent with the complaint that the product does not work being the main point of the sentence. The application shows the label only, so the confidence of these predictions is not known.

# CHAPTER 9
# CONCLUSION

## 9.1 Summary of the Project

The project Sentiment Analysis Using Ensemble Learning (ML) presents a machine learning-based system for classifying tweets into Positive, Negative, and Neutral sentiments. The Kaggle Twitter Entity Sentiment Analysis dataset is preprocessed and transformed into numerical features using Word-level and Character-level TF-IDF. A Random Forest classifier is used for sentiment prediction, while a hybrid rule-based approach supports the final classification. A Flask web interface enables users to enter text and view the predicted sentiment. The system demonstrates the practical application of machine learning and natural language processing for automated opinion analysis. However, challenges such as sarcasm, mixed opinions, noisy labels, and duplicate records may affect prediction reliability.

## 9.2 Achievement of Objectives

The main objectives, including dataset preparation, text preprocessing, feature extraction, Random Forest training, sentiment prediction, hybrid analysis, and web interface development, were achieved (Table 9.1).

**Table 9.1:** Objectives and Their Status

| Objective | Status |
|---|---|
| Dataset collection | Achieved – 74,682 records collected; 61,121 cleaned records used |
| Preprocessing | Achieved – seven-step cleaning pipeline |
| Feature extraction (TF-IDF) | Achieved – 15,000 combined features |
| Word and character features | Achieved – 10,000 word + 5,000 character features |
| Model training | Achieved – Random Forest with 100 trees |
| Prediction of unseen text | Achieved – 90.94% test accuracy; example sentences classified as expected |
| Hybrid improvement | Implemented – separate effect of the rule-based layer not measured |
| Interface | Achieved – Flask web interface |

## 9.3 Main Findings

- The Random Forest on combined Word and Character TF-IDF features achieved an accuracy of 90.94% on 12,225 unseen test tweets.
- Precision, recall and F1-score are 0.91 on average, and all three classes have F1-scores between 0.90 and 0.91.
- The system classified the example sentences as expected, including a mixed-sentiment sentence.
- Combining Word-level and Character-level TF-IDF gives a feature set of 15,000 values that captures both word patterns and spelling variations in informal text.
- The hybrid design keeps training inexpensive and the results easy to explain, and the Flask interface lets anyone test the model without programming knowledge.

## 9.4 Current Limitations

The system uses a single Random Forest classifier with a rule-based layer and does not compare multiple machine learning algorithms. The interface displays sentiment labels without confidence scores. Sarcasm, irony, and mixed sentiment remain challenging, and dataset noise may affect reliability. Since the dataset focuses on video-gamerelated tweets, performance on other domains requires further testing.

In addition, the reported accuracy comes from a single 80/20 split of one dataset, without repeated runs or crossvalidation, and the separate contribution of the rule-based layer was not measured. These points do not change the main result, but they should be kept in mind when the figures are compared with those of other studies.

The project nevertheless meets its aim: a tweet can be entered in a browser and classified as Positive, Negative or Neutral within a moment, using a model that was trained on more than sixty thousand labelled tweets and that reaches about 91% accuracy on tweets it has never seen.

## 9.5 FUTURE SCOPE

- Comparing Random Forest with Logistic Regression, Naive Bayes and Support Vector Machines, and developing voting or stacking ensembles [12], [13].
- Adding prediction confidence scores to the web interface.
- Removing duplicate records and applying cross-validation.
- Exploring transformer-based models such as BERT [15] and convolutional neural networks [14].
- Using SHAP or LIME to improve interpretability.
- Adding batch processing, real-time social-media analysis and public deployment.

Overall, the project establishes a foundation for automated sentiment analysis. Further experimentation and evaluation can improve its accuracy, robustness, interpretability, and applicability across different domains.

# CHAPTER 11 REFERENCES


1. B. Pang and L. Lee, “Opinion mining and sentiment analysis,” Foundations and Trends in Information Retrieval, vol. 2, no. 1–2, pp. 1–135, 2008.
2. B. Liu, Sentiment Analysis and Opinion Mining. Morgan & Claypool Publishers, 2012.
3. B. Pang, L. Lee and S. Vaithyanathan, “Thumbs up? Sentiment classification using machine learning techniques,” in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2002, pp. 79–86.
4. A. Go, R. Bhayani and L. Huang, “Twitter sentiment classification using distant supervision,” CS224N Project Report, Stanford University, 2009.
5. C. J. Hutto and E. Gilbert, “VADER: A parsimonious rule-based model for sentiment analysis of social media text,” in Proc. Int. AAAI Conf. Web and Social Media (ICWSM), vol. 8, no. 1, 2014, pp. 216–225.
6. K. Spärck Jones, “A statistical interpretation of term specificity and its application in retrieval,” Journal of Documentation, vol. 28, no. 1, pp. 11–21, 1972.
7. C. Cortes and V. Vapnik, “Support-vector networks,” Machine Learning, vol. 20, no. 3, pp. 273–297, 1995.
8. T. G. Dietterich, “Ensemble methods in machine learning,” in Multiple Classifier Systems (MCS 2000), LNCS vol. 1857, Springer, 2000, pp. 1–15.
9. L. Breiman, “Bagging predictors,” Machine Learning, vol. 24, no. 2, pp. 123–140, 1996.
10. L. Breiman, “Random forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.
11. Y. Freund and R. E. Schapire, “A decision-theoretic generalization of on-line learning and an application to boosting,” Journal of Computer and System Sciences, vol. 55, no. 1, pp. 119–139, 1997.
12. D. H. Wolpert, “Stacked generalization,” Neural Networks, vol. 5, no. 2, pp. 241–259, 1992. 13. Z.-H. Zhou, Ensemble Methods: Foundations and Algorithms. Chapman & Hall/CRC, 2012.
14. Y. Kim, “Convolutional neural networks for sentence classification,” in Proc. EMNLP, 2014, pp. 1746–1751.
15. J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in Proc. NAACL-HLT, 2019, pp. 4171–4186.
16. F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
17. S. Bird, E. Klein and E. Loper, Natural Language Processing with Python. O’Reilly Media, 2009.
18. Pallets Projects, “Flask documentation.” [Online]. Available: https://flask.palletsprojects.com
19. “Twitter Entity Sentiment Analysis,” Kaggle dataset. [Online]. Available: https://www.kaggle.com. [Dataset author, full link and access date to be inserted by the project team.]

# APPENDIX A SOURCE CODE

The source code files were not supplied with the project materials. The project team should insert here, with short explanations, the important code from the main files: preprocess.py (text preprocessing), train_model.py (Word and Character TF-IDF, Random Forest training and saving with Joblib), predict.py (loading the model and predicting, including the rule-based analysis) and app.py with the HTML templates (Flask interface). Source code to be inserted.